

Final Exam Preparation Guide

Programming and Data Structures — Week 15

Dr. Innocent Nyalala

Exam format

The final exam consists of three question types:

1. **Complexity analysis** — short-answer questions asking for the time or space complexity of a named algorithm or data structure operation, and, critically, the *reason* for that bound, not only the memorized bound itself (for example: quickselect’s expected $O(n)$ bound comes from the geometric-series argument in Week 14, not simply from “it’s like quicksort but faster”).
2. **Hand-trace questions** — trace a named algorithm step by step on a small, given input, exactly as done throughout every lecture in this course.
3. **Choose-the-right-tool scenarios** — given a new, unfamiliar problem description, identify which data structure or algorithm applies and justify the choice against at least one plausible alternative, following the reasoning process demonstrated in the Week 15 course review lecture’s two worked walkthroughs.

What to review, by module

Weeks	Topic	Key things to be able to do
2	Arrays and dynamic arrays	Explain amortized $O(1)$ append; compute a resizing trace
3	Stacks and queues	Implement using arrays or linked lists; identify LIFO vs FIFO use cases
4	Linked lists	Singly/doubly/circular; trace the Josephus problem
5-6	Sorting	Compare $O(n^2)$ vs $O(n \log n)$ methods; explain the comparison-sort lower bound

Weeks	Topic	Key things to be able to do
7	Binary trees	BST operations; AVL rotation triggers and types
8	Hash tables and heaps	Collision resolution; heap operations; priority queue merge
9	Dynamic programming	Identify overlapping subproblems; build a DP table by hand
10	Greedy algorithms	State and check the greedy-choice property; exchange argument sketch
11	Graph fundamentals	BFS vs DFS; cycle detection; topological sort (two methods)
12-13	Shortest paths and MSTs	Dijkstra's vs A* vs Bellman-Ford (when each applies); Prim's vs Kruskal's
14	Parallel and randomized algorithms	Amdahl's Law; Las Vegas vs Monte Carlo; quickselect's expected time

Practice strategy

1. Revisit each week's **Quiz and Exercise slides first** — these were specifically written to test the one distinction that lecture introduced.
2. For every algorithm covered, write from memory: its time complexity, one trade-off it makes against an alternative from the course, and a hand trace on a small example.
3. Practice the choose-the-right-tool reasoning process on 2-3 problems you have not seen before, not only ones from lecture or lab material. Two sample scenarios:
 - A ride-sharing app finding the fastest route with a known heuristic available (see Week 15's course review lecture for the worked answer).
 - A streaming median query problem (see Week 15's course review lecture for the worked answer).
4. Partial credit is given for demonstrating the reasoning process, even when a final numeric answer contains a minor error — showing why a technique was chosen and what was ruled out matters as much as the final answer.

A final note

Every worked example across all fifteen weeks of this course was independently verified against real running code before being presented, and several real errors were caught and corrected this way (see, for example, Week 12-13's Prim's-versus-Kruskal's verification slide). Bring that same discipline into the exam: when a hand-traced answer seems off, re-derive it rather than guessing.